

PlaySDK Unity Integration Guide (MatchmakingClient / MultiplayerClient)

- 1. [File List and Feature Overview](#)
- 2. [Quick Overview](#)
- 3. [Essential IDs & Setup](#)
- 4. [Core Interfaces & Configuration](#)
- 5. [MatchmakingClient Integration](#)
- 6. [MultiplayerClient Integration](#)
- 7. [Minimal Viable Integration](#)

File List and Feature Overview

Before starting integration, ensure your Unity project contains the following core scripts, libraries, and assets from the exported `.unitypackage`:

Core Scripts and Libraries

File/Folder	Type	Description	Required
Assets/Scripts/PlaySDK/MatchmakingClient.cs	C# Script	Core matchmaking client for room creation, joining, and game start flow	 Yes
Assets/Scripts/PlaySDK/MultiplayerClient.cs	C# Script	Core multiplayer client for real-time synchronization and game control	 Yes
Assets/Scripts/PlaySDK/MatchmakingDemoManager.cs	C# Script	Reference demo showing MatchmakingClient usage with UI integration	 Reference
Assets/Scripts/PlaySDK/MultiplayerDemoManager.cs	C# Script	Reference demo showing MultiplayerClient usage with all modules	 Reference
Assets/Plugins/WebGL/play-sdk.jslib	JS Library	JavaScript bridge between Unity and VIVERSE SDK (WebGL only)	 Yes (WebGL)
Assets/Plugins/NativeWebSocket/	C# + JS Library	WebGL WebSocket support	 Yes (WebGL)
Assets/Plugins/Newtonsoft.Json/Newtonsoft.Json.dll	C# Library	JSON serialization/deserialization library	 Yes

Supporting Files (Optional but Recommended)

File	Type	Description	Purpose
------	------	-------------	---------

File	Type	Description	Purpose
<code>Assets/Scripts/PlaySDK/Types.cs</code>	C# Script	Data structures (Actor, Room, RoomInfo, etc.)	Type definitions
<code>Assets/Scripts/PlaySDK/Modules/</code>	C# Scripts	General, NetworkSync, ActionSync, Leaderboard, Game modules	Feature modules
<code>Assets/Scripts/PlaySDK/Services/</code>	C# Scripts	Non-WebGL proxy client and supporting services	Non-WebGL support
<code>Assets/Scripts/PlaySDK/*DemoManager.cs</code>	C# Scripts	Additional demo managers for each module	Learning reference

Import Steps

Step 1: Download and Import UnityPackage

1. Download the `PlaySDK_v2.unitypackage` from your VIVERSE repository or distribution point
2. In Unity Editor, go to **Assets → Import Package → Custom Package...**
3. Select `PlaySDK_v2.unitypackage`
4. Review the file list (all files should be checked by default)
5. Click **Import**

✓ After import, verify:

- `Assets/Scripts/PlaySDK/` folder exists, containing `MatchmakingClient.cs` and `MultiplayerClient.cs`
- `Assets/Plugins/WebGL/` contains `play-sdk.jslib`
- `Assets/Plugins/NativeWebSocket/` contains WebSocket support
- `Assets/Plugins/Newtonsoft.Json/` contains `Newtonsoft.Json.dll`

Step 2: Configure Your Project Settings (WebGL Build)

If building for WebGL:

1. Go to **File → Build Settings**
2. Select **WebGL** as the target platform
3. In **Player Settings → Compression Format**, choose an appropriate format (Brotli recommended for smaller builds)

Step 3: Set Up App ID Configuration

1. Create a configuration script or add to an existing initialization script:

```
// Example: Assets/Scripts/GameConfig.cs
public class GameConfig
{
    public static readonly string APP_ID = "example-app-id"; // Replace with
your actual App ID
}
```

2. Store your **App ID** from <https://studio.viverse.com/upload>
3. Ensure this ID is accessible when initializing `MatchmakingClient`

Step 4: Optional - Import TextMesh Pro Essentials

If your UI uses TextMesh Pro:

1. Go to **Window → TextMesh Pro → Import TMP Essentials**
2. Click **Import** to add required fonts and materials
3. This ensures all sample UI displays correctly

What's Included in the Package

✔ **Full Source Code**

- Matchmaking and multiplayer client implementations
- All module implementations (General, NetworkSync, ActionSync, Leaderboard, Game)
- Demo managers with complete UI integration examples

✔ **JavaScript Bridge**

- `play-sdk.jslib` for WebGL communication with VIVERSE SDK
- Handles callbacks, promises, and message routing
- Automatically loads VIVERSE SDK from CDN: <https://www.viverse.com/static-assets/viverse-sdk/1.3.3/index.umd.cjs>

✔ **Demo Scenes (Optional)**

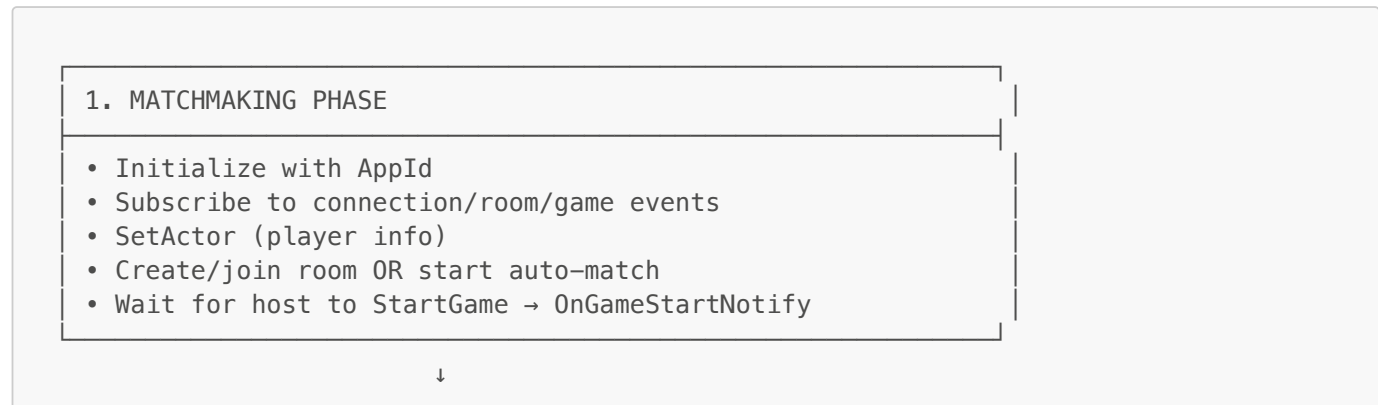
- `Assets/Scenes/MatchmakingDemo.unity` - Shows room creation, joining, auto-match flow
- `Assets/Scenes/MultiplayerDemo.unity` - Shows in-game sync, chat, leaderboard

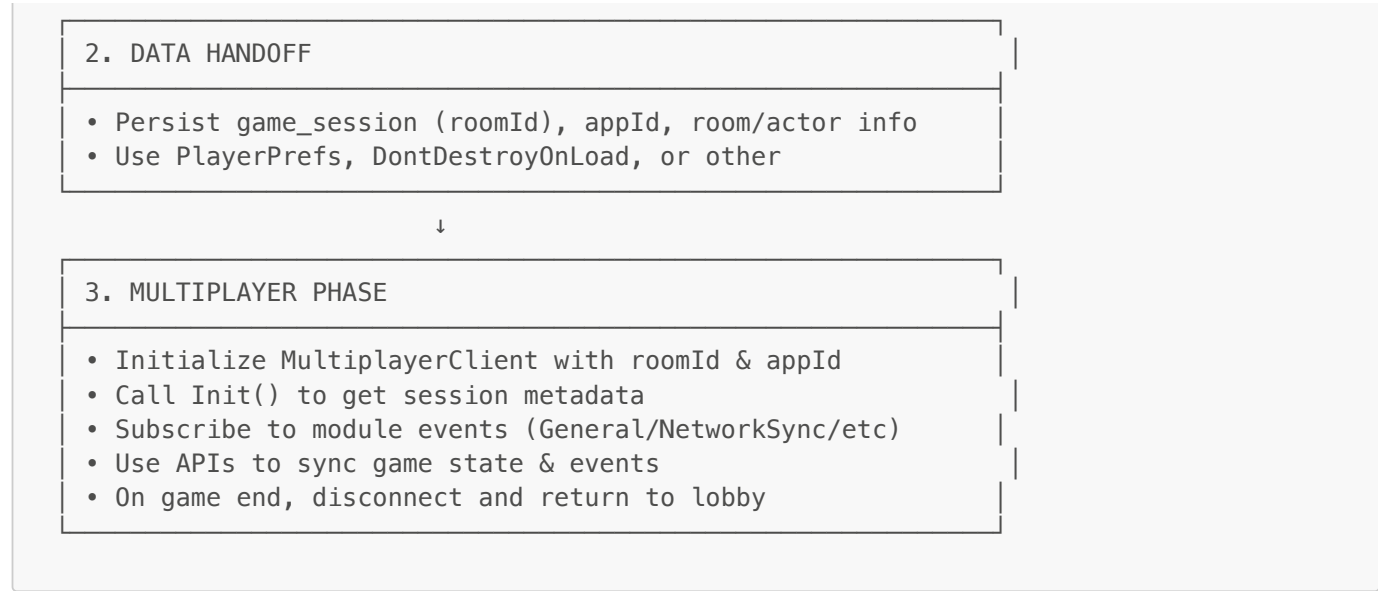
Minimum Files Required for Integration

Use Case	Required Files
Matchmaking only	<code>MatchmakingClient.cs</code> , <code>Types.cs</code> , <code>play-sdk.jslib</code> (WebGL)
Multiplayer only (if you have roomId already)	<code>MultiplayerClient.cs</code> , <code>Modules/*</code> , <code>Types.cs</code> , <code>play-sdk.jslib</code> (WebGL)
Full integration (Matchmaking → Multiplayer)	All core files listed above
Editor testing (non-WebGL)	<code>MatchmakingClient.cs</code> , <code>MultiplayerClient.cs</code> , <code>Services/MediasoupProxyClient.cs</code> , <code>Types.cs</code>

Quick Overview

Integration Flow





Two Main Clients

Phase	Client	Purpose	Key Methods
Matchmaking	MatchmakingClient	Find/create room, set player info, start when ready	Initialize, SetActor, CreateRoom, JoinRoom, StartGame
Multiplayer	MultiplayerClient	Real-time sync, chat, leaderboard, game control	Initialize, Init, NetworkSync.UpdateMyPosition, SendMessage, Leaderboard.LeaderboardUpdate

Essential IDs & Setup

1.1 Application ID (App ID)

What it is: Unique identifier for your game/app in the VIVERSE platform.

How to get it:

1. Go to <https://studio.viverse.com/upload>
2. Click "Create New World" (if new) or select existing
3. Navigate to "Manage Content" → "Overview" tab
4. Copy your **App ID**

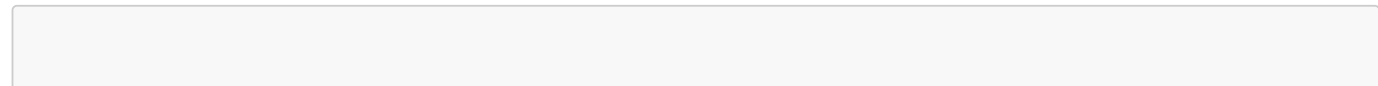
Where to use:

- MatchmakingClient.Initialize(appId)
- MultiplayerClient.Initialize(roomId, appId)

Core Interfaces & Configuration

2.1 MatchmakingClient Configuration

Attach MatchmakingClient to a GameObject and configure:



```
var go = new GameObject("MatchmakingClientInstance");
var mm = go.AddComponent<PlaySDK.MatchmakingClient>();
```

Key Properties:

Property/Method	Type	Description
<code>Initialize(appId)</code>	void	Initialize connection with your App ID. Call once.
<code>SetActor(actor)</code>	Task	Set your player info (name, properties, session_id).
<code>CreateRoom(name, mode, maxPlayers, minPlayers)</code>	Task	Create a room. Caller becomes host.
<code>JoinRoom(roomId)</code>	Task	Join an existing room.
<code>GetAvailableRooms()</code>	Task	List all joinable rooms. Triggers <code>OnRoomListUpdate</code> .
<code>LeaveRoom()</code>	Task	Leave current room. Triggers <code>OnJoinedLobby</code> .
<code>StartGame()</code>	Task	Host only. Start the game → others get <code>OnGameStartNotify</code> .

Key Events:

Event	Payload	When	Action
<code>OnConnect</code>	none	Successfully connected	Enable actor/room UI
<code>OnDisconnect</code>	none	Disconnected	Reset UI, allow reconnect
<code>OnJoinRoom</code>	Room	Joined a room	Show room info, hide create/list panels
<code>OnJoinedLobby</code>	none	Left room, back to lobby	Show create/join panels
<code>OnRoomListUpdate</code>	List	Room list refreshed	Update dropdown/list UI
<code>OnRoomActorChange</code>	List	Players in room changed	Refresh player roster
<code>OnGameStartNotify</code>	none	Game starting (you're in room)	Persist roomId/appld, switch scene
<code>OnError</code>	string	Any error occurred	Display error to user
<code>OnStateChange</code>	ConnectionState	State changed	Update UI state indicator

2.2 MultiplayerClient Configuration

Attach `MultiplayerClient` to a `GameObject` (keep alive across scenes):

```
var go = new GameObject("PlaySDKUnityBridge");
var mp = go.AddComponent<PlaySDK.MultiplayerClient>();
DontDestroyOnLoad(go); // Optional: keep across scenes
```

Key Methods:

Method	Type	Description
<code>Initialize(roomId, appId)</code>	Task	Initialize with room ID from matchmaking phase.
<code>Init()</code>	Task	Get session metadata (session_id, room_id, app_id).
<code>Disconnect()</code>	void	Cleanly disconnect.

Key Module Properties (access after `Init()`):

Module	Key Methods	Purpose
General	<code>SendMessage(text)</code>	Broadcast text messages to all players.
NetworkSync	<code>UpdateMyPosition(pos)</code> , <code>UpdateEntityPosition(id, pos)</code>	Sync entity positions.
ActionSync	<code>Competition(actionType, actionValue, actionId)</code>	Trigger and sync game actions.
Leaderboard	<code>LeaderboardUpdate(score)</code>	Update and track player scores.
Game	<code>TriggerGameStart()</code> , <code>TriggerGameEnd()</code> , <code>GetRoomInfo()</code> , <code>TriggerGameRestart()</code>	Game lifecycle control.

Key Events:

Event	Payload	When	Action
<code>OnConnected</code>	none	Connected to multiplayer service	Enable features, show ready
<code>OnClientConnected</code>	object	Another player joined room	Update roster; check "can start"
			⚠ Note: May not work properly in Unity Editor (requires WebRTC Proxy support), but works correctly in WebGL builds
<code>OnClientDisconnected</code>	object	A player left room	Update roster; recompute state
			⚠ Note: May not work properly in Unity Editor (requires WebRTC Proxy support), but works correctly in WebGL builds
<code>OnMessage</code>	object	Message from any player	Display chat/announcement
<code>NetworkSync.OnNotifyPositionUpdate</code>	object	Position update for entity	Spawn or update entity position
<code>NetworkSync.OnNotifyRemove</code>	object	Entity/player leaving	Remove entity from scene

Event	Payload	When	Action
ActionSync.OnCompetition	object	Action event from sync	Trigger corresponding game logic
Leaderboard.OnLeaderboardUpdate	object	Leaderboard rank change	Update scoreboard UI
Game.OnGameEnd	none	Game ended	Show results, offer return to lobby
Game.OnCountdownToStart	object	Countdown to game start	Lock input, show timer
Game.OnCountdownToEnd	object	Countdown to game end	Warn players, show timer
Game.OnGameRestart	none	Game restarting	Reset scene state
Game.OnErrorNotify	object	Game-side error	Display error and recovery option

MatchmakingClient Integration

3.1 Create and Initialize

```
public class MatchmakingManager : MonoBehaviour
{
    private PlaySDK.MatchmakingClient _matchmakingClient;

    public void ConnectToMatchmaking(string appId)
    {
        var go = new GameObject("MatchmakingClientInstance");
        _matchmakingClient = go.AddComponent<PlaySDK.MatchmakingClient>();

        // Subscribe to key events first
        _matchmakingClient.OnConnect += HandleConnect;
        _matchmakingClient.OnError += HandleError;
        _matchmakingClient.OnGameStartNotify += HandleGameStart;

        // Then initialize
        _matchmakingClient.Initialize(appId);
    }

    private void HandleConnect()
    {
        Debug.Log("[Matchmaking] Connected! Now set actor and create/join room.");
    }

    private void HandleError(string error)
    {
        Debug.LogError($"[Matchmaking] Error: {error}");
    }

    private void HandleGameStart()
    {
        Debug.Log("[Matchmaking] Game starting! Switching to multiplayer scene...");
    }
}
```

```
        // Save room data, switch scene (see section 3.4)
    }
}
```

3.2 Set Actor (Player Info)

```
public async void SetMyActor(string playerName)
{
    var actor = new PlaySDK.Actor
    {
        session_id = System.Guid.NewGuid().ToString(),
        name = playerName,
        properties = new Dictionary<string, object>
        {
            { "level", 1 }
        }
    };

    await _matchmakingClient.SetActor(actor);
    Debug.Log($"[Matchmaking] Actor set: {actor.name}");
}
```

3.3 Room Operations

Create Room

```
public async void CreateRoom(string roomName, string modeName, int minPlayers, int
maxPlayers)
{
    await _matchmakingClient.CreateRoom(
        name: roomName,
        mode: modeName,
        maxPlayers: maxPlayers,
        minPlayers: minPlayers
    );
    Debug.Log($"[Matchmaking] Room created: {roomName}");
}
```

List and Join Rooms

```
public async void RefreshRoomList()
{
    await _matchmakingClient.GetAvailableRooms();
    // Event OnRoomListUpdate will fire with updated list
}

public async void JoinRoom(string roomId)
{
    await _matchmakingClient.JoinRoom(roomId);
    // Event OnJoinRoom will fire on success
}
```


Leave Room

```
public async void LeaveRoom()
{
    await _matchmakingClient.LeaveRoom();
    // Event OnJoinedLobby will fire
}
```

Get Room Actors

```
public async void GetRoomActors()
{
    await _matchmakingClient.GetMyRoomActors();
    // Event OnRoomActorChange will fire with actor list
}
```

Start Game (Host Only)

```
public async void StartGame()
{
    if (_matchmakingClient.GetCurrentRoom()?.host_id == MY_SESSION_ID)
    {
        await _matchmakingClient.StartGame();
        // Event OnGameStartNotify fires for all in room
    }
    else
    {
        Debug.LogWarning("Only host can start game.");
    }
}
```

3.4 Handoff to Multiplayer Scene

When **OnGameStartNotify** fires, persist the necessary data and switch scene:

```
private void HandleGameStart()
{
    var room = _matchmakingClient.GetCurrentRoom();
    var appId = "your-app-id"; // or store it as a field

    if (room != null)
    {
        // Save for multiplayer scene
        PlayerPrefs.SetString("MultiplayerRoomId", room.game_session);
        PlayerPrefs.SetString("MultiplayerAppId", appId);
        PlayerPrefs.SetString("CurrentActorName",
            _matchmakingClient.GetCurrentActor()?.name ?? "Unknown");
        PlayerPrefs.Save();

        Debug.Log($"[Matchmaking] Switching to MultiplayerScene with roomId:
{room.game_session}");
    }
}
```

```

        UnityEngine.SceneManagement.SceneManager.LoadScene("MultiplayerScene");
    }
}

```

MultiplayerClient Integration

4.1 Initialize and Setup

Init() Method Parameters (MultiplayerInitOptions)

The `Init()` method supports an optional `MultiplayerInitOptions` parameter to control which game modules are enabled:

```

public class MultiplayerInitOptions
{
    public ModulesConfig modules { get; set; }
}

public class ModulesConfig
{
    public ModuleOption game { get; set; } // Game lifecycle management
    public ModuleOption networkSync { get; set; } // Position synchronization
    public ModuleOption actionSync { get; set; } // Action synchronization
    public ModuleOption leaderboard { get; set; } // Leaderboard
}

public class ModuleOption
{
    public bool enabled { get; set; } // Enable/disable this module
    public string desc { get; set; } // Module description
    // ... other configuration parameters
}

```

⚠ Important Notes:

- **Only enabled modules will function.** If `enabled = false`, the corresponding events will not be triggered
- **Recommendation:** If your game logic doesn't need a module (e.g., no leaderboard), set `enabled = false` to save resources
- If you don't provide `Init()` parameters, all modules will be enabled by default

Module Selection Guide

The 4 game modules (`Game`, `NetworkSync`, `ActionSync`, `Leaderboard`) are preset implementations designed for specific scenarios with limited flexibility (e.g., fixed position sync format, action format, and scoring logic).

If your game logic is complex or doesn't fit these preset formats (e.g., need to sync velocity/rotation, complex state machines, custom scoring rules), **we strongly recommend using `OnMessage` and `SendMessage()` with custom protocols**. This gives you full control over data format and flow, avoiding module constraints.

Initialization Example

```

public class MultiplayerManager : MonoBehaviour
{
    private PlaySDK.MultiplayerClient _multiplayerClient;

    async void Start()
    {
        string roomId = PlayerPrefs.GetString("MultiplayerRoomId", "default_room");
        string appId = PlayerPrefs.GetString("MultiplayerAppId", "app-default");

        var go = new GameObject("PlaySDKUnityBridge");
        _multiplayerClient = go.AddComponent<PlaySDK.MultiplayerClient>();

        // Subscribe to connection events
        _multiplayerClient.OnConnected += HandleMultiplayerConnected;
        _multiplayerClient.OnClientConnected += HandleClientConnected;
        _multiplayerClient.OnClientDisconnected += HandleClientDisconnected;

        // Initialize connection
        await _multiplayerClient.Initialize(roomId, appId);

        // Configure modules based on game requirements
        // Example: Enable only game and position sync, disable leaderboard and
        action sync
        var initOptions = new PlaySDK.MultiplayerInitOptions
        {
            modules = new PlaySDK.ModulesConfig
            {
                game = new PlaySDK.ModuleOption { enabled = true, desc = "Game
lifecycle" },
                networkSync = new PlaySDK.ModuleOption { enabled = true, desc =
"Position sync" },
                actionSync = new PlaySDK.ModuleOption { enabled = false }, //
Not needed for this game
                leaderboard = new PlaySDK.ModuleOption { enabled = false } //
Not needed for this game
            }
        };

        // Get session metadata and initialize selected modules
        var roomInfo = await _multiplayerClient.Init(initOptions);
        Debug.Log($"[Multiplayer] Initialized. RoomId: {roomInfo.room_id},
SessionId: {roomInfo.session_id}");

        // Subscribe to module events after Init
        SubscribeToModuleEvents();
    }

    private void HandleMultiplayerConnected()
    {
        Debug.Log("[Multiplayer] Connected! Game ready.");
    }

    private void HandleClientConnected(object data)
    {
        Debug.Log($"[Multiplayer] Client connected: {data}");
    }

    private void HandleClientDisconnected(object data)
    {

```

```

        Debug.Log($"[Multiplayer] Client disconnected: {data}");
    }

    private void SubscribeToModuleEvents()
    {
        // ⚠ Important: Only subscribe to events from enabled modules
        // If a module is not enabled in Init(), its events will not be triggered

        // General messages (always available, not affected by module configuration)
        _multiplayerClient.OnMessage += data =>
        {
            Debug.Log($"OnMessage: {data}");
        };

        // Network sync (requires networkSync module to be enabled)
        _multiplayerClient.NetworkSync.OnNotifyPositionUpdate += data =>
        {
            Debug.Log($"[NetworkSync] Position update: {data}");
        };
        _multiplayerClient.NetworkSync.OnNotifyRemove += data =>
        {
            Debug.Log($"[NetworkSync] Entity removed: {data}");
        };

        // Action sync (requires actionSync module to be enabled)
        _multiplayerClient.ActionSync.OnCompetition += data =>
        {
            Debug.Log($"[ActionSync] Action: {data}");
        };

        // Leaderboard (requires leaderboard module to be enabled)
        _multiplayerClient.Leaderboard.OnLeaderboardUpdate += data =>
        {
            Debug.Log($"[Leaderboard] Update: {data}");
        };

        // Game lifecycle (requires game module to be enabled)
        _multiplayerClient.Game.OnGameEnd += () =>
        {
            Debug.Log("[Game] Game ended!");
        };
        _multiplayerClient.Game.OnCountdownToStart += data =>
        {
            Debug.Log($"[Game] Starting in: {data}");
        };
        _multiplayerClient.Game.OnGameRestart += () =>
        {
            Debug.Log("[Game] Restarting...");
        };
        _multiplayerClient.Game.OnErrorNotify += data =>
        {
            Debug.LogError($"[Game] Error: {data}");
        };
    }
}

```

4.2 Module APIs

General (Chat/Messages)

```
_multiplayerClient.SendMessage("Hello, team!");
```

NetworkSync (Position/Entity Sync)

```
// Update your position
_multiplayerClient.NetworkSync.UpdateMyPosition(new { x = 1.5f, y = 2.0f, z = 3.5f,
w = 0 });

// Update another entity
_multiplayerClient.NetworkSync.UpdateEntityPosition("enemy_001", new { x = 10, y =
5, z = 8, w = 0 });
```

ActionSync (Game Actions)

```
// Fire an action: actionType, actionValue, actionId
_multiplayerClient.ActionSync.Competition("shoot", "fireball", "action_001");
```

Leaderboard

```
// Update your score
_multiplayerClient.Leaderboard.LeaderboardUpdate(score: 1050);
```

Game (Lifecycle & Control)

```
// Start game
_multiplayerClient.Game.TriggerGameStart();

// Update game state
_multiplayerClient.Game.TriggerGameUpdate(totalPlayer: 4);

// End game
_multiplayerClient.Game.TriggerGameEnd();

// Restart
_multiplayerClient.Game.TriggerGameRestart();

// Get current room info
var roomInfo = await _multiplayerClient.Game.GetRoomInfo();
```

4.3 Disconnect and Cleanup

```
void OnDestroy()
{
```

```

    if (_multiplayerClient != null)
    {
        _multiplayerClient.Disconnect();
        Debug.Log("[Multiplayer] Disconnected and cleaned up.");
    }

    // Clear cached data
    PlayerPrefs.DeleteKey("MultiplayerRoomId");
    PlayerPrefs.DeleteKey("MultiplayerAppId");
    PlayerPrefs.Save();
}

```

Minimal Viable Integration

5.1 Two-Scene Minimal Setup

Scene 1: MatchmakingScene

Create a GameObject named **MatchmakingBootstrap** and attach this script:

```

using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MinimalMatchmakingBootstrap : MonoBehaviour
{
    private PlaySDK.MatchmakingClient _mm;
    [SerializeField] private string appId = "example-app-id"; // Replace with your
App ID

    async void Start()
    {
        var go = new GameObject("MatchmakingClientInstance");
        _mm = go.AddComponent<PlaySDK.MatchmakingClient>();

        _mm.OnConnect += () => Debug.Log("[MM] Connected");
        _mm.OnError += err => Debug.LogError($"[MM] Error: {err}");
        _mm.OnGameStartNotify += OnGameStart;

        _mm.Initialize(appId);

        // Create actor
        var me = new PlaySDK.Actor
        {
            session_id = Guid.NewGuid().ToString(),
            name = $"Player{UnityEngine.Random.Range(1000, 9999)}",
            properties = new Dictionary<string, object> { { "level", 1 } }
        };
        await _mm.SetActor(me);

        // Create room
        await _mm.CreateRoom($"{me.name}'s Room", "Room", maxPlayers: 4, minPlayers:
2);
    }
}

```

```

private void OnGameStart()
{
    var room = _mm.GetCurrentRoom();
    if (room != null)
    {
        PlayerPrefs.SetString("MultiplayerRoomId", room.game_session);
        PlayerPrefs.SetString("MultiplayerAppId", appId);
        PlayerPrefs.Save();
        SceneManager.LoadScene("MultiplayerScene");
    }
}

```

Scene 2: MultiplayerScene

Create a GameObject named **MultiplayerBootstrap** and attach this script:

```

using UnityEngine;
using System.Threading.Tasks;

public class MinimalMultiplayerBootstrap : MonoBehaviour
{
    private PlaySDK.MultiplayerClient _mp;

    async void Start()
    {
        string roomId = PlayerPrefs.GetString("MultiplayerRoomId", "test_room");
        string appId = PlayerPrefs.GetString("MultiplayerAppId", "app-test");

        var go = new GameObject("PlaySDKUnityBridge");
        _mp = go.AddComponent<PlaySDK.MultiplayerClient>();

        _mp.OnConnected += () => Debug.Log("[MP] Connected");

        await _mp.Initialize(roomId, appId);
        var info = await _mp.Init();
        Debug.Log($"[MP] Ready. SessionId: {info.session_id}");

        // Start syncing
        _mp.NetworkSync.UpdateMyPosition(new { x = 0, y = 0, z = 0, w = 0 });
    }

    void OnDestroy()
    {
        if (_mp != null)
        {
            _mp.Disconnect();
        }
    }
}

```

Troubleshooting & Common Issues

SetActor Response is Empty

- **Cause:** JS callback not triggering properly; `SendMessage` not available.
- **Fix:** Ensure `PlaySDK_UnityGameObject_Name` and `PlaySDK_UnityCallback_MethodName` are correctly initialized in `play-sdk.jslib`.

"Already Closed" Error on MultiplayerClient

- **Cause:** Trying to reconnect without clearing the old instance.
- **Fix:** In `play-sdk.jslib`, set `PlaySDK_MultiplayerClient_Instance = null` after `disconnect()`.

onClientConnected Parsing Error

- **Cause:** JS sends a string instead of an object.
- **Fix:** Wrap the data in an object in `play-sdk.jslib`:

```
instance.onClientConnected((data) =>
  sender('onClientConnected', { user_session_id: data })
);
```

Room/Match Not Starting

- **Cause:** Conditions not met (host not set, player count mismatch, room closed, etc.).
- **Fix:** Check `OnRoomActorChange` events; verify host status and player count before calling `StartGame()`.

References

Demo Scenes & Scripts:

- `Assets/Scenes/MatchmakingDemo.unity` + `Assets/Scripts/PlaySDK/MatchmakingDemoManager.cs`
- `Assets/Scenes/MultiplayerDemo.unity` + `Assets/Scripts/PlaySDK/MultiplayerDemoManager.cs`

Core SDK Files:

- `Assets/Scripts/PlaySDK/MatchmakingClient.cs`
- `Assets/Scripts/PlaySDK/MultiplayerClient.cs`
- `Assets/Plugins/WebGL/play-sdk.jslib`
- VIVERSE SDK (loaded automatically from CDN: <https://www.viverse.com/static-assets/viverse-sdk/1.3.3/index.umd.cjs>)

Documentation:

- VIVERSE Studio: <https://studio.viverse.com/upload>

End of Guide